

### Esercizio A1

È facile notare che l'algoritmo `Merge()` proposto non è altro che una versione di Insertion Sort dove vengono specificati gli indici di inizio e di fine.

Il caso ottimo corrisponde, come nell'InsertionSort, al caso in cui il vettore sia già ordinato. In questo caso, tutte le volte che viene applicata `Merge()`, il costo è lineare e quindi l'equazione di ricorrenza è:

$$T(n) = \begin{cases} 2T(n/2) + n & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Utilizzando il Master Theorem, il costo computazionale risultante è pari a  $\Theta(n \log n)$ .

Il caso pessimo corrisponde, come nell'InsertionSort, al caso in cui il vettore sia ordinato in senso opposto. In questo caso, tutte le volte che viene applicata `Merge()`, il costo è quadratico e quindi l'equazione di ricorrenza è:

$$T(n) = \begin{cases} 2T(n/2) + n^2 & n > 1 \\ 1 & n \leq 1 \end{cases}$$

Utilizzando il Master Theorem, il costo computazionale risultante è pari a  $\Theta(n^2)$ .

### Esercizio A2

Le regole di colorazione degli Alberi RedBlack sono le seguenti:

1. La radice è nera
2. Tutte le foglie sono nere
3. Entrambi i figli di un nodo rosso sono neri
4. Tutti i cammini semplici da un nodo  $u$  ad una delle foglie contenute nel sottoalbero radicato in  $u$  hanno lo stesso numero di nodi neri

Il codice seguente è lo scheletro della soluzione. Innanzitutto, viene verificata la proprietà (1), che è facile da controllare. Poi, si invoca una funzione ricorsiva `blackHeight()` che analizza le restanti proprietà degli alberi RB e ritorna l'altezza nera dell'albero, in caso le proprietà siano rispettate, oppure -1, se l'albero non rispetta le proprietà.

---

```
boolean isRedBlack(TREE T)
```

---

```
  if T.color == RED then
    |   return false
  else
    |   return (blackHeight(T) > 0)
```

---

La funziona ricorsiva che verifica le proprietà (2)-(4) sull'intero albero è la seguente.

- La proprietà (2) è verificata sui nodi **nil** e assumendo che essi siano nodi speciali, in cui sia comunque determinare il colore. Se questo è rosso, si ritorna -1, ad indicare che in un qualche punto dell'albero una delle proprietà non è rispettata.
- La proprietà (3) viene verificata su tutti i nodi rossi, controllando se il proprio genitore (se esiste) ha anch'esso colore rosso.
- La proprietà (4) viene verificata calcolando l'altezza nera dei rami sinistro e destro del nodo attuale. Se una di esse è negativa, le proprietà (2)-(4) non sono verificate nel sottoalbero relativo. Se sono diverse, la proprietà (4) non è rispettata e si ritorna -1. Altrimenti, si restituisce l'altezza nera del nodo attuale.

---

```

int blackHeight(TREE T)
    % Proprietà (2)
    if T == nil then
        | return iif(T.color == RED, -1, 1)

    % Proprietà (3)
    if t.color == RED and t.parent ≠ nil and t.parent.color == RED then
        | return -1

    % Proprietà (4)
    int bhL = blackHeight(T.left)
    int bhR = blackHeight(T.right)
    if bhL < 0 or bhR < 0 or bhL ≠ bhR then
        | return -1
    else
        | return bhL + iif(t.color == BLACK, 1, 0)

```

---

La complessità è  $\Theta(n)$ , in quanto `blackHeight()` implementa una visita in profondità.

### Esercizio A3

Se ordiniamo il grafo tramite un ordinamento topologico, otteniamo uno dei tanti ordinamenti possibili. Può capitare che un esame con una propedeuticità appaia prima di un esame senza propedeuticità. Tuttavia sappiamo che quando un esame viene estratto dallo stack, tutti gli esami che sono propedeutici ad esso sono stati già estratti.

Non siamo interessati alla distanza minima; invece, siamo interessati alla distanza massima a partire da tutti i nodi che non hanno archi entranti.

La distanza viene inizializzata a zero per tutti i nodi. Tutte le volte che viene estratto un nodo, la sua distanza massima effettiva è già stata calcolata (in quanto quando viene estratto, tutti gli esami propedeutici sono stati estratti). A questo punto, possiamo aggiornare la distanza di tutti i nodi adiacenti, incrementandola se necessario.

Al termine, ritorniamo la distanza massima trovata incrementata di uno, perché stiamo calcolando il numero di sessioni.

---

```

minSessions(GRAPH G)
    STACK S = topsort(G)
    int[] dist = new int[1...G.n]
    foreach u ∈ G.V() do
        | dist[u] = 0
    while not S.isEmpty() do
        | NODEu = S.pop()
        | foreach v ∈ G.adj(u) do
            | | dist[v] = max(dist[v], dist[u] + 1)
    return max(dist) + 1

```

---

La complessità è quella di due visite in profondità, ovvero  $O(m + n)$ .